



SkillIssue – Projektleírás

Neumann János Gimnázium, Technikum és Kollégium

Csörgő Csaba, Flender Dávid

2025. december

Tartalomjegyzék

1. Bevezetés	3
1.1. A projekt célja	3
1.2. Inspiráció és motiváció	3
1.3. Csapattagok és szerepkörök	3
1.3.1. Felelőségek	3
1.3.2. Projektszervezés	3
1.3.3. Verziókezelés	4
2. Követelmények	5
2.1. Funkcionális követelmények	5
2.2. Nem-funkcionális követelmények	6
2.3. Használati módok (felhasználó, adminisztrátor, vendég)	7
3. Rendszerarchitektúra	8
3.1. Áttekintés	8
3.2. Szolgáltatások és felelőségeik	8
3.3. Kommunikáció a szolgáltatások között	8
3.4. Architektúra diagram	9
4. Technológiák	10
4.1. Backend - Laravel	10
4.2. WebSocket - Node.js	11
4.3. Frontend - Vue.js	11
4.4. Konténerizáció - Docker	12
4.5. Adatbázis-terv	13
4.5.1. ER Diagram	13
4.5.2. Entitások leírása	13
4.5.3. Kapcsolatok és megszorítások	14
5. Backend - API	15
5.1. Autentikáció	15
5.2. Felhasználókezelés	15
5.3. Kérdéskezelés	15
5.4. Meccs és kör logika	16
5.4.1. Egyjátékos mód	16
5.4.2. Többjátékos (Rangsorolt 1 vs. 1) mód	16
5.5. Bejelentő rendszer	17
5.5.1. Játékos bejelentés	17
5.5.2. Kérdés bejelentés	17
5.5.3. Kérdések elbírálása	17
5.6. Service-to-service kommunikáció	17
6. WebSocket szerver	18
6.1. Miért külön szolgáltatás	18
6.2. Kapcsolat az API-val	18
6.3. Valós idejű meccs folyamat	18

7. Frontend	20
7.1. Oldalak és navigáció	20
7.2. Komponensstruktúra	20
7.3. Állapotkezelés	20
7.4. WebSocket integráció kliens oldalon	20
7.4.1. Konfiguráció	20
7.4.2. WebSocket kapcsolat	21
7.4.3. Eseménykezelés	21
8. Játékmechanika	22
8.1. Egyjátékos mód	22
8.2. Rangsorolt 1 vs. 1 mód	22
8.3. Matchmaking folyamat	23
8.4. Pontozás és rangsor	24
9. Fejlesztési környezet és üzembe helyezés	25
9.1. Fejlesztői stack felállítása	25
9.2. Környezeti változók	26
9.3. Fejlesztői vs. Éles konfiguráció	27
9.4. Makefile parancsok	27
9.5. Tesztelési stratégia	28
9.6. Manuális tesztelés	28
9.7. Automatizált tesztelés	28
9.7.1. Backend tesztek (Laravel / PHPUnit)	28
9.7.2. Frontend tesztek (Vue 3 / Vitest)	28
10.Összefoglalás	30
10.1. Megvalósított funkciók	30
10.2. Nehézségek és tanulságok	30
10.3. További fejlesztési lehetőségek	31
A. Környezeti változók dokumentációja	32
B. Makefile parancsok referencia	33

1. fejezet

Bevezetés

1.1. A projekt célja

A SkillIssue nevű projekt célja, hogy megvalósítson egy olyan platformot, amely az érettségi vizsga előtt álló diákok számára biztosít lehetőséget eredményes és ingyenes felkészülésre. A webalkalmazás lehetőséget biztosít a fejlődés nyomon követésére, szintlépés, illetve rangrendszer formájában.

Elsődleges célcsoportunk a fiatal, érettségi vizsga előtt álló diákok, azonban alkalmazásunk lehetőséget nyújt azok számára is, akik szeretnék felfrissíteni tudásukat, vagy az életük későbbi szakaszában készülnek érettségi vizsgatételre. A SkillIssue gamifikált kialakítása lehetővé teszi, hogy az alkalmazást az érettségi felkészülésen túl bármilyen más célra is használják - akár hobbiból, akár önfejlesztés céljából.

1.2. Inspiráció és motiváció

Érettségi vizsgára felkészüléskor észrevettük, hogy nincs jelenleg olyan alkalmazás magyar nyelven, a magyar érettségi követelményekre szabva, amely lehetővé tenné, hogy játékos formában készüljenek a diákok a vizsgatételre. Ez motiválja a csapatot a SkillIssue projekt megvalósítására.

Ígletet merít a projekt hasonló, játékosított elemeket tartalmazó alkalmazásokból, mint például a Duolingo¹ és a GeoGuessr². Ezeket az ismereteket kombinálja a tapasztalatokkal, hogy olyan alkalmazást készítsen, amely egyaránt érdekes és élvezetes, de emellett oktató jellegű is.

1.3. Csapattagok és szerepkörök

A projektet két fős csapat valósítja meg: Csörgő Csaba és Flender Dávid.

A projektcsapat tagjainak kiválasztása azon alapul, hogy a csapat tagjai korábban már sikeresen megvalósították a gördülékeny munkavégzést más, különböző jellegű projekteken is.

1.3.1. Felelősségek

A projektcsapat tagjai az alkalmazás minden részében egyaránt szerepet vállaltak, de fő szerepvállalásuk az alábbiak szerint csoportosítható:

Csörgő Csaba Fő felelőssége a DevOps és a Backend munkavégzési folyamatok, főként az autentikáció, és a WebSocket működése.

Flender Dávid Fő felelőssége a design kialakítása, a WebSocket funkciók frontend oldali integrálása és a megfelelő végfelhasználói élmény biztosítása.

1.3.2. Projektszervezés

A projekt szervezése egy közös kanban táblán történt. Erre a célra a Trello alkalmazás került kiválasztásra. A feladatokat a következő oszlopokra lehet osztani: To Do, In Progress, Blocked, Review, Done, Released.

¹Nyelvtanuló alkalmazás nagy felhasználóbázissal, amely játékos formában nyújt lehetőséget a tanulásra.

²Google Maps-re épülő játékal alkalmazás, többjátékos móddal, rangsorolt játékmóddal. Lényege: megtalálni a Google Street View lokációt a térképen.

1.3.3. Verziókezelés

A verziókezelést a Git alkalmazás biztosítja, amely megkönnyíti a csapatmunkát és a párhuzamos munkavégzést. A Git mellett a projekt a GitHub felhőszolgáltatást is használja.

A fejlesztés során szigorú elnevezési konvenciókat követ a csapat a repository átláthatósága érdekében. A branch-ek neveit a feladat jellegétől függően az alábbi előtagokra lehet bontani:

- **feature/**: Új funkciók implementálása.
- **bugfix/**: Hibák javítása.
- **refactor/**: Kód újrastrukturálása és optimalizálása.
- **main**: A stabil, tesztelt és kiadásra kész állapotú kód ága.

A commit üzenetek esetében is hasonló következetességre törekszik a csapat. Minden beküldött módosítás tartalmazza a változtatás típusát, például: **Feature:** [leírás], **Fix:** [leírás] vagy **Refactor:** [leírás]. Ez a strukturált megközelítés lehetővé teszi a projekt történetének gyors és egyértelmű visszakövethetőségét.

2. fejezet

Követelmények

2.1. Funkcionális követelmények

Felhasználó

- **Autentikáció:** A felhasználó tud regisztrálni és bejelentkezni
- **Bejelentkezések tárolása:** A felhasználó bejelentkezéseinek tárolása
- **Jogkörök:** Felhasználó jogkörének kezelése: felhasználó, adminisztrátor, vendég

Előrehaladás

- **Rangrendszer:** Elo¹ pontrendszerhez hasonló rangrendszer megvalósítása (többjátékos módhoz)
- **Tapasztalati pontok:** XP² eltárolása minden felhasználóhoz, aminek értéke minden helyes válaszadással nő

Játékmenet

- **Egyjátékos játékmód:** Egyjátékos gyakorló játékmód, feleletválasztós kérdésekkel
- **Többjátékos játékmód:** Többjátékos rangsorolt játékmód, feleletválasztós kérdésekkel, 1 vs. 1 formátumban
- **Meccskeresés:** Két játékos összesorolása, rang alapján

Bejelentések

- **Kérdés bejelentés:** Lehetőség bejelenteni hibásnak vélt kérdéseket
- **Felhasználó bejelentése:** Lehetőség bejelenteni ellenfelet, amennyiben csalást feltételez a felhasználó

²Elo (Élő) pont: Prof. Élő Árpád által kifejlesztett pontszámítási rendszer, amely leginkább a sakkban terjedt el, de napjainkban számos online rangsorolt játékmóddal rendelkező játék használja.

²XP: Tapasztalati pont – egy olyan játékmechanikai változó, amely minden játszma után rendszerint n értékkel növekszik, és nem csökkenhet.

2.2. Nem-funkcionális követelmények

Teljesítmény

- Az oldalak betöltésének csökkentése, Lazy Loading és Eager Loading alkalmazásával
- Lehetőség szerinti legkevesebb harmadik féltől származó NPM csomagok felhasználása

Biztonság

- Jelszavak titkosított tárolása
- Cookie alapú session autentikáció
- Service-to-service kommunikáció titkosított tokennel (WebSocket és API között)
- Érvényes SSL/TLS tanúsítvány használata
- Minimális jogosultságok Docker konténerekben belül

Használhatóság

- Modern böngészők támogatása
- Reszponzív design
- HTTP Polling lehetősége, amennyiben a WebSocket nem támogatott a böngésző által

Üzemeltetés

- Docker segítségével könnyen üzembe helyezhető webalkalmazás létrehozása
- Fejlesztői és éles környezet támogatása, elkülönített konfigurációval

Statisztika, csalás elleni intézkedések

- Backend a legkevesebb adatot küldi a kliensnek
- Felhasználó pontosságának rögzítése
- Válaszadás gyorsaságának rögzítése
- JWT tokenek használata a játékmenet során az adatok integritásának védelmére (kérdés- és meccstokenek)

Megbízhatóság

- A játszma állapotának megőrzése felhasználó kilépése esetén
- Játzma elhagyása esetén ne szakadjon meg a játékmenet a másik félnek

2.3. Használati módok (felhasználó, adminisztrátor, vendég)

Vendég

Nem bejelentkezett felhasználó

- Főoldal megtekintése
- Bejelentkezés, regisztráció
- Játékos profilok megtekintése

Felhasználó

Bejelentkezett felhasználó, normál jogkör

- Főoldal megtekintése
- Játékos profilok megtekintése
- Egyjátékos, többjátékos (rangsorolt) módok
- Kérdés, ellenfél bejelentése

Adminisztrátor

Bejelentkezett felhasználó, emelt jogkör

- Mindazt, amit a felhasználó
- Kérdések, válaszok hozzáadása, törlése, szerkesztése
- Kérdés -és játékos bejelentések kezelése, bírálása
- Felhasználó kitiltása

3. fejezet

Rendszerarchitektúra

3.1. Áttekintés

Ez a fejezet az éles (production) környezet felépítését taglalja.

A rendszer 4 szolgáltatásból épül fel:

Adatbázis MySQL 8.0.36

PHP-FPM API (Laravel) kiszolgálása a server-kliens kommunikáció megvalósításához

WebSocket Node.js és Socket.IO a valós idejű játékmenet megteremtéséhez

nginx Frontend kiszolgálása, WebSocket Proxy és FastCGI átjáró biztosítása.

3.2. Szolgáltatások és felelősségeik

Adatbázis	Biztonságos adattárolás, gyors adatelérés a Laravel EloquentORM használatával. Tranzakciók kezelése, a konzisztencia és adat integritás megőrzése érdekében.
API	Biztosítja a server és kliens közötti kommunikációt. Lehetővé teszi az üzleti logika tárolását, végrehajtását. Biztonságos autentikáció megvalósítása, HTTP végpontok elérésének biztosítása a kliens számára. Adatbázis-elérés biztosítása az EloquentORM használatával.
WebSocket	A valós idejű játékmenet megteremtése. Real time szinkronizáció a Frontend és a Backend között. Megbízható kommunikációs csatorna kiépítése, központosított ideiglenes adattárolás (meccskeresés, aktív játszmák). A WebSocket üzleti logikát nem tartalmaz, és adatbázis-kezelést nem végez.
nginx	Frontend kiszolgálása, SSL/TLS kapcsolat kiépítése. WebSocket proxy üzemeltetése valamint FastCGI átjáró a PHP-FPM szolgáltatásnak az api.skillissue.hu virtuális hoszton keresztül.

3.3. Kommunikáció a szolgáltatások között

A rendszer egy közös Docker hálózatba van bekapcsolva. Ez lehetővé teszi, hogy a szolgáltatások egymást könnyedén elérhessék, a szolgáltatás nevére hivatkozva.

Nginx útvonalválasztás

Az nginx server (1.25) két porton (80, 443) hallgat. Amikor kérés érkezik, megpróbálja kiépíteni a HTTPS (SSL/TLS) kapcsolatot a kliens és a server között.

Két virtuális hosztot szolgál ki:

Frontend: a skillissue.hu frontendet teszi elérhetővé. Emellett be van állítva egy proxy is, a `/socket.io` útvonalra, amely a 3000-res porton hallgató Socket.IO WebSocket szerverre irányít.

API: az api.skillissue.hu API-t teszi elérhetővé, amely a FastCGI átjárón keresztül jut el a PHP-FPM szolgáltatáshoz. A PHP-FPM szolgáltatásnak nincsen elérhető portja a külvilág felé, csak a FastCGI vonalon elérhető, csakis az nginx-en keresztül.

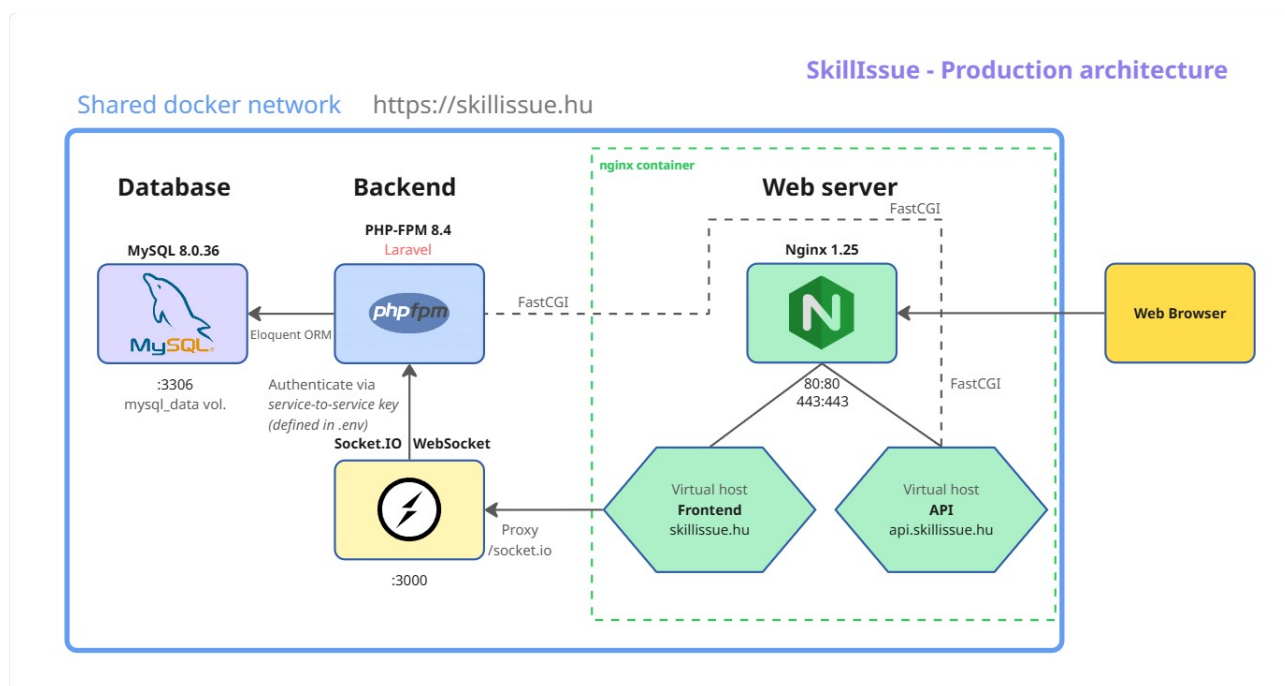
Backend és adatbázis

A Laravel backend PHP-FPM 8.4 alatt fut. Az adatbázis és Laravel közötti kommunikáció az EloquentORM segítségével valósul meg. A MySQL 8.0.36-os verziója a 3306-os porton hallgat, csakis a belső hálózaton, ezáltal kívülről nem elérhető az API megkerülésével. Az adatok tárolására Docker Volume van létrehozva.

API és WebSocket

A WebSocket és API közötti kommunikációhoz service-to-service token van létrehozva. Ez a kulcs egy közös `.env` fájlban van tárolva, így mindkét szolgáltatás hozzáfér. Ez a token az `Authorization` fejlécben kerül elküldésre Bearer tokenként. Ez biztosítja, hogy kizárólag a WebSocket szerver kommunikálhasson közösen az API-val.

3.4. Architektúra diagram



3.1. ábra. A SkillIssue éles környezetének architektúrája

4. fejezet

Technológiák

4.1. Backend - Laravel

A Backend oldalon a megvalósítás a PHP nyelvre épül, azon belül is a **Laravel** keretrendszer köré.

Autentikáció A Sanctum csomag használatával valósul meg az autentikációs réteg. Előnye, hogy egyszerűen használható, lightweight megoldás. Mivel a SkillIssue alkalmazás csakis webes megvalósítással rendelkezik, Cookie alapú session autentikáció mellett döntöttünk. Az API a REST elveit követi, kivéve az autentikációs réteget, ahol a bejelentkezett felhasználó session-je tárolódik.

EloquentORM ORM használata megkönnyíti a fejlesztői munkafolyamatot, azáltal, hogy migration-ökben tárolódnak az adatbázis táblák sémái, és osztályok példányain érhetőek el az entitások. Kiküszöböli az SQL injection veszélyeit, mivel nem nyers SQL utasításokra épül a rendszer.

PHP-FPM/FastCGI Gyors, skálázható kiszolgálási forma az API számára

MySQL Azért esik a MySQL adatbázisra a választás, mert gyors lekérdezéseket és könnyű integrációt biztosít a Laravel-lel. Más lehetséges megoldások, például a SQLite, azért nem kerül alkalmazásra, mivel kevésbé skálázható adatbázis megoldásokat biztosít.

JWT tokenek Az alkalmazás a JWT tokeneket nem autentikációhoz használja, hanem 3 különböző tokent képes kiállítani a megfelelő lekérdezések alkalmával: Kérdés token, egyjátékos meccs token, és rangsorolt (1 vs. 1) meccs token. Ezekre a megoldásokra azért van szükség, hogy kódolt formában tudjon adatot továbbítani a kliens számára, és ne plaintext formában küldjön fontos adatokat. Továbbá, a JWT tokenek működési elvéből adódóan könnyen kezelhető a tokenek lejáratási ideje, így például lehetőség van a megoldás beküldésének elutasítására, amennyiben a kérdés token lejár.

4.2. WebSocket - Node.js

A WebSocket megvalósítása a Socket.IO csomaggal történik. Az említett csomagnak az előnye, hogy könnyen kiépíthető a WebSocket kapcsolat, emellett lehetőséget biztosít HTTP Polling fallback-re, amennyiben a kliens nem képes WebSocket kapcsolatot kezdeményezni.

A WebSocket megvalósítása nem tartalmaz üzleti logikát, és adatbázis módosítást nem végez. Ezek helyett az API-val kommunikál.

WebSocket → API kapcsolat A WebSocket és a Laravel API közös környezeti változókból dolgozik. Ez teszi lehetővé, hogy egy közös token-t tudnak olvasni, amelyet service-to-service tokenként nevezünk. Ez egy konstans token, amely nem tárol semmilyen adatot, csupán validációra szolgál, mivel az összes `/api/internal/`-al kezdődő útvonalat csakis a WebSocket veheti igénybe, felhasználó közvetlenül nem.

Felhasználó autentikáció A WebSocket külön, belső API útvonalon autentikálja a felhasználót. Ennél a folyamatnál kihasználja azt, hogy a sütik minden lekérésnél továbbmennek a lekérés helyére. Ezáltal a cookie-kat manuálisan elküldve egy belső útvonalnak, könnyedén kinyerhetők a felhasználó adatai.

Állapotkezelés Az ideiglenes állapotkezelésre a legegyszerűbb módot alkalmazza a rendszer, így Hash Map formájában tárolja a különböző adatokat, amelyek a következők: Meccskeresési queue, megerősítésre váró meccsek¹ és folyamatban lévő meccsek. Ennek a tárolási módszernek a legnagyobb hátránya, hogy ha a WebSocket szolgáltatás váratlanul újraindul, a folyamatban lévő meccsek elvesznek, ezáltal nem befejezhetők. *Ilyenkor Elo változás nem történik.*

4.3. Frontend - Vue.js

A Frontend szolgáltatáshoz a Vite build tool-t alkalmazza a rendszer, ami fejlesztői környezetben lehetővé teszi a projekt gyors elindítását, fejlesztői webszerver futtat, és hot reload funkciót is biztosít.

Mivel a SkillIssue egy közepes méretű projekt, a Vue.js egy gyors, egyszerű és átlátható fejlesztési folyamatot biztosít. Más keretrendszerek, például az Angular azért nem kerül alkalmazásra, mert a projekt méretéhez képest indokolatlanul komplex megoldás lenne.

Útvonalválasztás A routing problémáját a Vue Router oldja meg, ami a standard megoldás egy Vue alkalmazás esetében. Lehetővé teszi, hogy átirányítson, URL paramétereket fogadjon, és metaadatot kapcsoljon bizonyos oldalakhoz (például vendég elérheti-e).

Állapotkezelés A prop drilling elkerülése érdekében a Pinia csomagot alkalmazza a rendszer, ami egy Vue.js-re szabott állapotkezelő megoldás. A Pinia került alkalmazásra a projekt megvalósítása során, mivel ez az egyik legkönnyebben használható store csomag a Vue ökoszisztémában. Emellett a hivatalos Vue dokumentáció is a Pinia Store-t javasolja, illetve jól integrálható a Vue Dev Tools kiegészítővel is.

WebSocket kliens oldalon A WebSocket kapcsolatot a Frontend bejelentkezéskor kezdeményezi, vagy már bejelentkezett felhasználó esetén akkor, amikor validálásra kerül a felhasználó bejelentkezett állapota. Ez teszi lehetővé, hogy az alkalmazás érzékelje a már megkezdett meccseket, és automatikusan visszacsatlakoztassa a felhasználót a folyamatban lévő többjátékos meccsbe. A meccskeresés és játékmenet elkülönítve, store-ban kerül tárolásra. Itt futnak azok a listener-ök is, amelyek figyelik a WebSocket felől érkező eseményeket.

HTTP Kérések Az API-hoz intézett kérések kezeléséhez az Axios csomagot alkalmazza a projekt. Ennek előnye, hogy megkönnyíti a HTTP kérések folyamatát a JavaScript-be beépített fetch API-hoz képest. Saját Axios példányt alkalmaz a projekt, ezáltal letisztultabb, átláthatóbb kódot biztosít, hiszen bizonyos szükséges fejlécek automatikusan kitöltésre kerülnek.

Design A projekt megvalósítása során felhasználásra került a Tailwind CSS keretrendszer, amely egyszerűbbé teszi az oldalon megjelenő elemek stílusának személyre szabását. A Tailwind kellő szabadságot biztosít a fejlesztőnek, hogy egyedi Design-t alkothasson, szemben olyan framework-ökkel, mint például a Bootstrap, ami sokkal szigorúbb keretek közé szorítja a design-t. A projekt egy letisztult, konzisztens, glassmorph elemekkel díszített arculatot valósít meg. A színpaletta élénk színeket tartalmaz, amely tovább fokozza az oldal játékos hangulatát. A Tailwind mellett a FontAwesome ikonkönyvtár ingyenes verziója lett alkalmazva, a nagy választék miatt. A FontAwesome CDN linken keresztül kerül beágyazásra.

¹A két játékosnak, akiket a Matchmaking rendszer összesorol, meg kell erősíteni a játszmát - Ne kezdődjön el úgy meccs, hogy egyik felhasználó nincs a számítógép előtt (AFK).

4.4. Konténerizáció - Docker

Döntés indoklása Két fős projektként a Docker megkönnyíti a fejlesztési folyamatot, mivel azonos környezeten tudnak dolgozni a csapattagok. Ezzel elkerülve azt a gyakori problémát, hogy az egyik számítógépen működik, a másikon pedig nem, például verziókülönbség miatt.

Megvalósítás A projekt Docker Compose-t használ, ezáltal könnyedén futtatható több szolgáltatás egyszerre, összehangolt módon. A két környezet (production, development) külön compose fájlokban van definiálva.

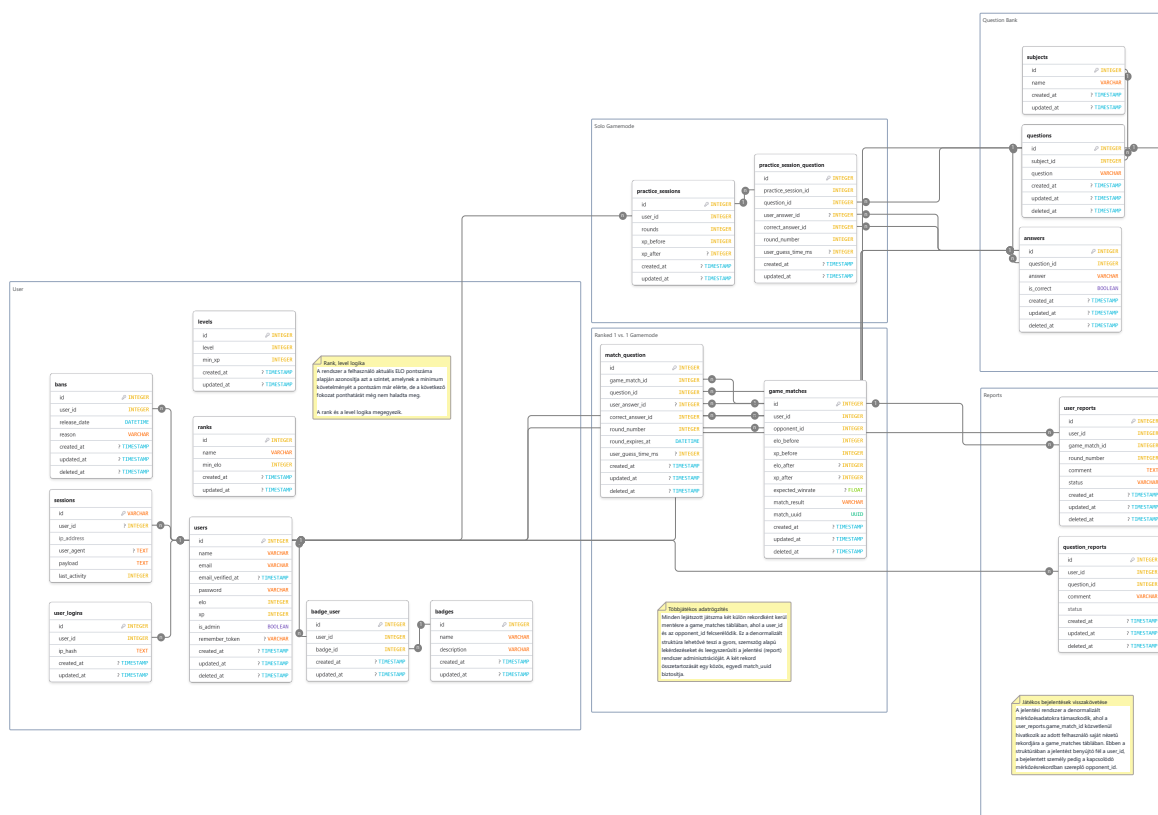
Fejlesztői környezet elvárásai Könnyen elindítható, fejlesztő-barát megoldás megvalósítása. A hot-reload támogatása és a host számítógép és a konténer közötti jogosultságok szinkronizálása, különösen az **artisan** parancssori eszköz esetében, hiszen az képes fájlokat létrehozni a host számítógépen. Ezek miatt szükséges a volume-ok kialakítása, amivel a hot-reload és a MySQL hosszú távú adattárolása megvalósulhat. A jogosultságok érdekében a host felhasználó Group ID és User ID értékeit használja a konténer is.

Éles (production) környezet elvárásai Egyszerű szállítás és deployment. Megoldásként a Makefile parancsai szolgálnak, amelyek lehetővé teszik, hogy egyszerűen egy paranccsal kiadható legyen egy új verzió. A másik fontos szempont a biztonság, ezért ahol csak lehetséges, non-root felhasználó van konfigurálva a konténerben. A gyors kiszolgálás és proxy-zás érdekében nginx webservert került alkalmazásra. Az nginx a Cloudflare origin SSL-t alkalmazza, a HTTPS kapcsolat kiépítésének érdekében. A konfiguráció továbbá lehetővé teszi, hogy ameddig az alkalmazás tesztelési fázisában van, jelszóval védett legyen a virtuális hoszt. További információ: 3.1. ábra

4.5. Adatbázis-terv

4.5.1. ER Diagram

Az adatbázis szerkezeti felépítését és az entitások közötti kapcsolatokat az alábbi ER (Entity-Relationship) diagram szemlélteti. A diagram a DrawDB eszközzel készült. Online elérhető ide kattintva.



4.1. ábra. A SkillIssue alkalmazás ER diagramja

4.5.2. Entitások leírása

A rendszer főbb entitásai és azok szerepkörei:

- **users**: A felhasználók alapvető adatait, kompetitív rangját (ELO), tapasztalati pontjait (XP) és adminisztrátori jogosultságait tárolja.
- **questions & answers**: A kvízzjáték magját alkotó kérdések és a hozzájuk tartozó helyes/helytelen válaszlehetőségek tárolóhelye, tantárgyak (subjects) szerint csoportosítva.
- **game_matches**: A felhasználók közötti párbajok adatait rögzíti. A gyors lekérdezhetőség és az adatbázis kapcsolatok egyszerűsítésének érdekében redundáns (denormalizált) módon tárolja a meccseket, minden résztvevő szemszögéből egy-egy rekorddal.
- **match_question**: A mérkőzések során feltett konkrét kérdéseket, a felhasználó válaszait és a válaszdíjakat rögzíti körökre bontva.
- **practice_sessions**: A gyakorló mód adatait tárolja, ahol a felhasználó tét nélkül (ELO változás nélkül) szerezhet tapasztalatot.
- **reports**: Felhasználó és kérdés jelentések kezelésére szolgáló entitások a platform moderációjához.
- **bans & logins**: Biztonsági entitások, amelyek a kitiltások és a bejelentkezési előzmények (IP hash) nyomon követésére szolgálnak.

4.5.3. Kapcsolatok és megszorítások

Az adatbázis integritását az alábbi főbb kapcsolatok és megszorítások biztosítják:

1. **Egy-a-többhöz (1:N) kapcsolatok:**

- Egy `subject` több `question`-höz kapcsolódik.
- Egy `question`-höz több `answer` tartozik.
- Egy `user` több mérkőzéssel (`game_matches`), gyakorlással (`practice_sessions`) és jelentéssel rendelkezhet.

2. **Több-a-többhöz (N:M) kapcsolatok:**

- A felhasználók és badge-ek kapcsolata a `badge_user` kapcsolótáblán keresztül valósul meg.

3. **Egyediségi megszorítások (Unique):** Az e-mail címek, a rangok minimum pontszámai (`min_elo`) és a szintek (`min_xp`) egyedisége biztosítja, hogy a rangsorolási és azonosítási rendszer egyértelmű maradjon.

4. **Denormalizációs stratégia:** A `game_matches` tábla szándékosan duplikált rekordokat tárol (minden meccset kétszer, felcserélt `user_id` és `opponent_id` mezőkkel), amit egy közös `match_uuid` fog össze. Ez egyszerűsíti a lekérdezési folyamatot több adatbázis kapcsolaton keresztül.

5. fejezet

Backend - API

5.1. Autentikáció

A SkillIssue a felhasználó autentikációjához a Sanctum csomagot használja, azon belül is cookie alapú session autentikációt. Ennek az az előnye, hogy a kérésekkel automatikusan elküldődik a süti, így könnyen azonosítható a felhasználó egyéb fejlécek megadása nélkül. A projekt megvalósítása webes felületen történik, ezért nincs szükség a JWT és egyéb autentikációra szolgáló tokenek sokoldalúságára, viszont a jövőbeli igények figyelembevételével később jó fejlesztési lehetőség lehet.

5.2. Felhasználókezelés

A felhasználókról a következő információk kerülnek tárolásra:

- Név
- Email
- Jelszó (Bcrypt hash)
- ELO
- XP
- Adminisztrátori jogosultsággal rendelkezik-e?
- Bejelentkezésekhez tartozó IP címek (hash-elt formában, csupán összehasonlításra)¹

A felhasználókat továbbá ki lehet tiltani a rendszerből, ez valójában egy új adatbázis-rekord létrehozását jelenti a **bans** táblában. Amennyiben a felhasználónak aktív kitiltása van, a rendszer nem engedi játszani, és tájékoztatja a kitiltás indokáról és annak lejáratáról.

5.3. Kérdéskezelés

A kérdéseket lekérni normál felhasználónak nincs jogosultsága, csak az adminisztrátoroknak. Az adminisztrátor jogosultsággal rendelkező felhasználók kérdéseket módosíthatnak, törölhetnek, vagy adhatnak hozzá - már létező, vagy új tantárgy megadásával.

Normál felhasználó csak akkor kérhet le kérdést, amennyiben aktív játékmenetben van (JWT tokennel azonosítva) és a `/api/questions/get-one` útvonalat használja. Ez egy véletlenszerűen választott kérdést ad vissza (amelyet a felhasználó még nem látott a meccs során) a válaszokkal együtt.

¹A rendszer a V1.0.0 verzióban nem használja fel a tárolt IP címeket semmilyen módon. Jövőbeli fejlesztési lehetőségek megvalósítására kerülnek tárolásra.

5.4. Meccs és kör logika

5.4.1. Egyjátékos mód

Játzsma folyamata lépésről lépésre:

1. **Indítás:** A játékos elindítja a játszmát. Ilyenkor az API felé kérést indít (POST `/practice-sessions`), ami elkészíti a játszmát az adatbázisban, illetve létrehozza a JWT token-t (*GameToken*), ami a játszma azonosítására szolgál az API és a Frontend kommunikációjában. A token eltárolja a felhasználó azonosítóját és az elkészített játszma azonosítóját. Amennyiben a felhasználó aktív kitiltás alatt áll, a játszma nem lesz létrehozva, és az API 403-as hibával tér vissza.
2. **Átírányítás:** A játékos átkerül egy új oldalra, amelyen az URL paraméterben a játszma JWT tokenje található meg. Ez biztosítja, hogy amennyiben elhagyja a játékot, vissza tud lépni, és nem veszti el a teljes előrehaladását.
3. **Kérdés lekérése:** A kör elején a Frontend POST kérést küld a `/api/questions/get-one` útvonalra, amely válaszol a véletlenszerűen kiválasztott kérdéssel, illetve az azokra eltárolt lehetséges válaszokat is visszaadja, összekevert sorrendben. A kérés csak akkor végződik sikeresen, ha a játszma elkészítésekor létrehozott JWT token el lett küldve, és megfelelő (felhasználó azonosító egyezik a kérést elküldő felhasználóéval). Új JWT token kerül kiállításra (*QuestionToken*), amely tartalmazza a kérdés és a felhasználó azonosítóját.
Amennyiben az adatbázisban nincs elegendő kérdés (*körök száma > kérdések száma*), a játszma csak addig folytatódik, ameddig el nem fogynak a felhasználó által (az adott játszmában) még nem látott kérdések; ezek után hibát ad.
4. **Válaszadás:** Amennyiben a játékos be kívánja küldeni a válaszát, ki kell választani az általa helyesnek vélt választ, majd az alsó "Beküldés" gombra kattintva elküldeni a szervernek. Ilyenkor POST kérés fog végbemenni a `/api/answers/verify/{válasz azonosító}` végpontra, mely kötelezően elvárja a QuestionToken-t és a GameToken-t. Amennyiben a válasz azonosító helyén a null literál szerepel, az eltárolt válasz is NULL értékű lesz, ez jelzi, hogy a felhasználó nem adott választ időben. A kérésre válaszként megérkezik, hogy a válasz helyes volt-e, illetve hogy mi lett volna a helyes válasz. Amennyiben a játékos az utolsó kérdésre válaszolt, visszaadja a megszerzett XP pontokat is ($n_{\text{helyes válaszok}} \cdot 10$). Amennyiben a felhasználó nem küldi be magától az idő leteltével a válaszát, a rendszer automatikusan beküldi a kiválasztottat - ha van olyan - egyébként üres választ küld be (null).

5.4.2. Többjátékos (Rangsorolt 1 vs. 1) mód

A többjátékos módhoz a lekéréseket a WebSocket szolgáltatás végzi, a belső `/api/internal` előtaggal ellátott végpontokra.

Ezek a végpontok a következők:

- `/game-matches`: A törzsben 2 paramétert kaphat: `user_a_id` és `user_b_id`. Amennyiben a felhasználók közül valamelyik jelenleg aktív kitiltás alatt van, a játszma nem lesz létrehozva, és 403-as hibakóddal tér vissza az API. Adatbázis tranzakcióban kezeli a meccs létrehozását a rendszer, ugyanis egy játszma két adatbázis rekordnak felel meg (Részletek: 4.5. szakasz). Ez kiküszöböli annak lehetőségét, hogy az egyik meccs létrehozásra kerüljön, a másik viszont sikertelen legyen. A JWT token payloadjában a következő adatok szerepelnek: Egyedi meccsazonosító, A felhasználó azonosítója és B felhasználó azonosítója. A rendszer továbbá kiszámolja a várt nyerési arányt a játékosoknak, a 8.4 szakasz alapján, majd visszaküldi a két felhasználó nevét, hogy a játékosok ismerhessék az ellenfelüket.
- `/questions/get-one`: Arra szolgál, hogy a WebSocket lekérhesse a következő kérdést, amit feltesz a játékosoknak. A rendszer kezeli, hogy olyan kérdéseket kapjanak a játékosok, amit még nem láttak az adott játszmában, valamint azt, hogy a válaszok sorrendje összekevert legyen. A rendszer azt is kezeli, hogyha tévesen bejönne még 1 kérés egy olyan meccsre, ami már véget ért (következő kör > maximum körök). Kiállít egy *QuestionToken*-t, a két felhasználó azonosítójával, valamint a kérdés azonosítójával. Tranzakcióban bejegyzi az adatbázisba a kérdéseket, a lejárat idejükkal együtt.
- `/answers/verify/{válasz azonosító}`: Arra szolgál, hogy visszaadja a helyes választ az adott kérdésre, bejegyezze a felhasználó válaszát és kezelje a játszma lezárását (ha az utolsó körben érkezik a kérés). A beküldő felhasználó azonosítója a kérés törzsében érkezik. A rendszer kezeli, ha olyan felhasználó küld be választ, aki nem vesz részt azon a játszmán, illetve azt is, ha nem küld be választ a felhasználó (NULL). A rendszer továbbá azzal is foglalkozik, ha a válasz nem az adott körben következő kérdéshez tartozik, továbbá figyelembe veszi a kérdés lejáratát is. Az adatbázisban a felhasználó válaszadási idejét is tárolja

a rendszer, amit a kérdés elkészítési időbélyegből és a szerveridőből számol. A rendszer nem korrigál a felhasználó hálózati késleltetése alapján, sem az adatbázis lekérésének sebességével - nagyságrendileg a valóságnak megfelelő értéket tárol. Amennyiben az utolsó válasz kerül beküldésre, a rendszer kiszámolja az eltárolt várható nyerési esély alapján az ELO nyereséget/veszteséget, majd visszaküldi azt, a helyes válasszal együtt.

Megjegyzés: A többjátékos módhoz tartozó `/api/internal` előtagú végpontok nem ugyanazokat a control-lereket használják, mint az egyjátékos módban, annak ellenére, hogy nevük ugyanaz. Az elnevezés oka a két játékmenet logikájának hasonlóságán alapszik.

5.5. Bejelentő rendszer

Két fajta bejelentést különböztet meg a rendszer: játékos bejelentés és kérdés bejelentés.

5.5.1. Játékos bejelentés

Bármelyik játékos bejelentheti az ellenfelét az 1 vs. 1 rangsorolt meccsek folyamán. Olyan személyek bejelentésére nincs lehetőség, akikkel nem játszott együtt a játékos. Az adatbázis tárolja a bejelentéseknél többek között azt, hogy melyik körben lett jelentve az ellenfél (a könnyebb kivizsgálás érdekében), de ezt a felhasználó felülírhatja, hogy melyik volt számára a leggyanúsabb kör. Emellett szöveges leírást is adhat az eseményről.

5.5.2. Kérdés bejelentés

Amennyiben a felhasználó egy adott kérdést és/vagy az azokra megadott válaszokat helytelennek találja, esetleg hibás "helyes válasz" van megjelölve, bejelentést tehet. Szöveges megjegyzés formájában leírhatja mi volt a probléma az adott kérdéssel.

5.5.3. Kérdések elbírálása

Az alábbi állítások mind a kérdés -és a játékos bejelentésekre igazak.

- Normál felhasználó csak POST kéréseket küldhet a végpontokra; más, illetve saját bejelentéseit nem tekintheti meg.
- Az adminisztrátor jogosultsággal rendelkező felhasználók egy külön felületen tekinthetik meg a beérkezett jelentéseket. Azok státuszát (nyitva, vizsgálat alatt, lezárva) módosíthatják, reportokat törölhetnek. Játékos bejelentés esetében látják a felhasználó statisztikáját is, kérdés bejelentésnél pedig a kérdést és a hozzá tartozó válaszokat. Az adminisztrátorok saját megjegyzésüket is odaírhatják a felhasználóé mellé, így jelezve a többi adminisztrátornak észrevételüket.

5.6. Service-to-service kommunikáció

A több szolgáltatáson keresztül való kommunikációk validálására az API az `EnsureServiceTokenIsValid` nevű middleware-t használja. Ennek feladata mindössze annyi, hogy összehasonlítsa a `.env` fájlban meghatározott kulccsal a fejlécben megadott Bearer token-t.

6. fejezet

WebSocket szerver

6.1. Miért külön szolgáltatás

A SkillIssue alkalmazás a WebSocket szervert külön szolgáltatásként definiálja. A cél elérésére a Socket.IO csomagot használja. Ennek okai az alábbiak:

- Az API-tól függetlenül skálázható megoldás.
- Amennyiben az API újraindul, a WebSocket szolgáltatás továbbra is elérhető marad. Emiatt a folyamatban lévő meccsek nem szakadnak meg, és a legtöbb esetben tovább folytathatók.
- Feladatkörök szétválasztása

6.2. Kapcsolat az API-val

Az API kommunikáció során az axios példányban meghatározott fejlécek kerülnek elküldésre, amelyekből kiemelten fontos az **Authorization** header. Ez tartalmazza a service-to-service token-t.

A felhasználó azonosításához egy middleware áll rendelkezésre a WebSocket-en belül, amely a kapott cookie-k alapján az API-val való kommunikáció után eltárolja a felhasználó adatait. Ezek után a kommunikáció a REST standard szerint történik.

6.3. Valós idejű meccs folyamat

A folyamat a következők szerint zajlik:

1. A felhasználó csatlakozik a WebSocket-hez. Amennyiben van aktív játszma hozzárendelve, a WebSocket elküldi ezt a kliensnek, ami automatikusan visszacsatlakoztatja.
2. Amennyiben nincs aktív meccs folyamatban, a felhasználó csatlakozhat a Matchmaking Queue-hoz. Ekkor bekerül egy hash map struktúrában tárolt sorba, amit a WebSocket 2 másodpercenként kiértékel (8.3 fejezet alapján).
3. Amennyiben sikeresen párosításra kerül 2 játékos, létrejön egy **Pending match**, amelyet mindkét játékosnak el kell fogadnia. Amennyiben elfogadja, csak akkor kerül át az **ongoingMatches** hash map-be, és kerül létrehozásra a játszma az adatbázisban.
4. Lekérésre kerül egy kérdés, majd ez megjelenítésre kerül a játékosok előtt (mindkét játékos ugyan azt a kérdést kapja).
5. Amennyiben egy játékos elküldi a válaszát, kiértékelésre kerül, de még nem látja meg, hogy helyesen válaszolt-e, mindaddig amíg a másik játékos el nem küldi válaszát. Ez idő alatt a WebSocket emit-el egy eseményt, ami által a frontend meg tudja jeleníteni a feliratot: *"Várakozás [Játékos neve] válaszára"*.
6. Amennyiben mindkét felhasználó válaszol, a WebSocket emit-eli a helyes választ, ami megjelenítésre kerül a felhasználók számára. Ezután rövid várakozással a folyamat újraindul a következő kérdéssel. Amennyiben nincs már több kérdés, tehát az utolsó kört játszották, az utolsó válasz beküldése után a játszma kiértékelésre kerül, és a felhasználónak ezt a WebSocket elküldi. A kiértékelést az API végzi, nem a WebSocket. Ilyenkor a felhasználót átirányítja az összegző felületre, ahol láthatja a meccset, és akár meg is tudja osztani másokkal az eredményt.

7. Kikerül a játszma az `OngoingMatches` hash map-ből.

Matchmaking Amennyiben a játékos már meccskeresési queue-ban van, nem léphet be újra. Ugyanez a helyzet, ha már van aktív játszma a felhasználónak.

WebSocket kapcsolat Amennyiben a felhasználó több lapról nyitotta meg a böngészőben a SkillIssue alkalmazást, csak az egyikben lesz aktív WebSocket kapcsolata, így a másiktól a real-time játékmenet nem fog működni.

7. fejezet

Frontend

A projekt Frontend megoldása Vue.js-en, annak is a modern Composition API-án alapul. Azért a Vue.js-re esett a választás más keretrendszerek helyett, mert a Vue olyan megoldásokat biztosít, amelyek egy kis-közepes projekt számára előnyösebbek. Könnyebb tiszta, jól átlátható és egyszerűen fenntartható kódot írni, mint egy komplexebb keretrendszerben (pl. Angular, React). Emellett a támogatása is kiemelkedő, olyan csomagokkal, mint például a Pinia és a Vue Router.

7.1. Oldalak és navigáció

A navigációhoz a Vue Router csomagot használja az alkalmazás. Több előnye is van ennek a megoldásnak:

1. Az oldalak nem töltődnek újra, csak a komponensek cserélődnek ki
2. Lehetőség van Eager- és Lazy loading alkalmazására is
3. Könnyen megoldható a jogosultságok ellenőrzése
4. Jól bevált ipari standard

A Vue Router-ben a `WebHistory` előzménymódot használja az alkalmazás, mivel ez ideális egy SPA számára.

7.2. Komponensstruktúra

A Frontend kódbázis mappastruktúrájának kialakításakor elsődleges szempont az átláthatóság és a könnyű kezelhetőség. Ennek érdekében a komponensek külön **components** könyvtárban helyezkednek el. Ezen könyvtáron belül altípusok mappái találhatóak, pl.: `Auth`, `Dashboard`, `Game`, `Generic`. A komponensek elkészítése során fő szempont a modularitás, így kiemelt szerepet kapnak a `prop`-ok a komponensek kiépítésében.

7.3. Állapotkezelés

A `prop-drilling` megakadályozására Pinia store-t használ a rendszer, amelyek a kódbázis **stores** könyvtárában helyezkednek el. A kialakításuk során fontos szempont, hogy a Single Responsibility Principle-nek megfeleljenek, azaz csak egy jól körülhatárolt feladatot végezzenek. A következő állapotkezelésre alkalmas store-ok találhatóak meg a projektben:

UserStore A felhasználó bejelentkeztetése, azonosítása, kijelentkeztetése, és adatainak tárolása.

MatchmakingStore WebSocket kommunikáció létesítése, események kezelése, meccskeresés elindítása, befejezése, átirányítás a játék oldalra találat esetén.

GameStore Minden játék közbeni esemény lekezelése, válasz beküldése, új kérdés fogadása, felhasználó visszatlatkoztatása folyamatban lévő játszma esetén.

7.4. WebSocket integráció kliens oldalon

7.4.1. Konfiguráció

Frontend oldalon az alkalmazás a Socket.IO kliensének NPM csomagját használja. Olyan módon van konfigurálva, hogy amennyiben a felhasználó böngészője támogatja a WebSocket kapcsolatokat, elsődlegesen azt használja, de fallback-ként HTTP polling-ra is képes.

7.4.2. WebSocket kapcsolat

A WebSocket kapcsolat akkor jön létre, amikor a felhasználó megnyitja az oldalt. Amennyiben több mint egy lapról van megnyitva a skillissue.hu oldal egy felhasználó alatt, a megfelelő működés csak az elsőként megnyitott lapon garantált. Kivétel ez alól az összes bejelentő oldal, amelynél külön kikapcsolásra kerül a WebSocket kapcsolat annak érdekében, hogy a bejelentő oldal a folyamatban lévő játékmenetet ne zavarja.

7.4.3. Eseménykezelés

A WebSocket kliensnek érkező eseményeket az adott eseményhez kapcsolódó Pinia store figyeli. Hiba esetén (pl. WebSocket szolgáltatás megáll) értesíti a felhasználót a rendszer, és leállítja a folyamatban lévő meccskeresést (amennyiben van ilyen).

8. fejezet

Játékmechanika

A rendszer alapvetően két játékformát valósít meg: Egyjátékos gyakorló mód és Rangsorolt többjátékos (1 vs. 1) mód.

8.1. Egyjátékos mód

Tét nélküli gyakorló játékmód. A felhasználó feleletválasztós kérdéseket kap, ahol csak egy jó válasz lehetséges. Adott időn belül válaszolnia kell a kérdésre. Amennyiben nem válaszol, üres válasz lesz eltárolva, és nem kap érte pontot. Válaszadás esetén (legyen az üres válasz, vagy a felhasználó által kiválasztott) a következő kérdés automatikusan lekérésre kerül, miután a rendszer megmutatta a helyes választ a felhasználónak. Ez a folyamat a konfigurációtól függően n körig folytatódik. Az utolsó kör végén a válasz beküldésével a rendszer összesíti a helyes válaszokat és ennek megfelelően XP-vel jutalmazza a játékost, ahol minden helyes válasz után 10 XP jár. Negatív végeredmény nincsen, a rendszer XP-t vagy bármi más pontot nem von le a játékostól.

8.2. Rangsorolt 1 vs. 1 mód

Rangsorolt játszma, azaz XP mellett ELO pontokat is szerezhethet a játékos; viszont itt már negatív eredmény is kerülhet számításra az ELO esetében.

A játékos belép a meccskeresési folyamatba (bővebben: 8.3. szakasz). Amikor a rendszer megfelelő ellenfelet talál a felhasználónak, el kell azt fogadnia, ezzel megerősíti, hogy a számítógép előtt tartózkodik, és készen áll a meccsre. Amennyiben nem erősíti meg valamelyik játékos, a meccskeresés számukra leáll, és jelzi a rendszer, hogy legalább az egyik játékos nem fogadta el a játszmat. Amennyiben mindkét játékos elfogadja, továbbjutnak a játék oldalra.

Az egyjátékos módhoz hasonlóan, feleletválasztós formában kapnak kérdéseket. Mindkét játékos ugyan azt a kérdést kapja, ugyan azokkal a válaszokkal. Szintén, egy helyes válasz lehetséges az adott kérdésnél. A játékosoknak megadott idő áll rendelkezésre, hogy beküldjék a válaszukat. Amennyiben az idő letelik, vagy mindkét felhasználó beküldi a választ az idő letelte előtt, a rendszer sárga színnel jelzi a felhasználó választ, lilával az ellenfél választ, és zölddel a helyes választ. Amennyiben a helyes válasz, a felhasználó válasza, és az ellenfél válasza közül legalább kettő ugyan az a válasz, a háttérszínük megoszlik, kettő vagy három részre. Például, ha a legelső volt a helyes válasz, és a felhasználó, valamint az ellenfél válasza is, akkor az első válasz háttérszínének első harmada sárga, második harmada lila, és utolsó harmada zöld háttérszínű lesz, így jelezve a felhasználónak, hogy mindenki a helyes választ küldte be.

Amennyiben az idő letelik, és van megjelölt, de még nem beküldött válasza valamelyik felhasználónak, a rendszer automatikusan beküldi azt. Ha nincs, üres válasz kerül beküldésre. Ilyenkor a következő kör automatikusan indul.

Az utolsó kör végén, amikor mindkét felhasználó beküldte a választ, vagy letelt az idő, a rendszer összegzi a játszmat, és kiszámítja a hozzáadott és levont pontokat, valamint a nyertest (Bővebben: 8.4. szakasz).

8.3. Matchmaking folyamat

A meccskeresés úgynevezett képességalapú párosítást használ (SBMM¹).

Működése: Konstans változók tárolják azokat az értékeket, amik befolyásolják, milyen a játszma, ami a leginkább unfair, de még az elfogadható tartományba tartozik. A következők kerülnek definiálásra:

Base elo range Az alapbeállítás, ekkora ELO különbségben keresi az ellenfelet a játékosnak a rendszer.

Maximum elo range A maximum elfogadható ELO különbség a két játékos között

Long wait time Az a másodpercben megadott időtartam, ami már "hosszú meccskeresésnek" minősül. A felhasználóbázis méretéhez érdemes beállítani.

Lépésről lépésre:

1. Növekvő sorba rendezi az összes queue-ban lévő felhasználót, ELO szerint.
2. A for ciklus végig iterál a queue hosszán, ahol mindig a jelenlegi és a sorban következő játékost keresi. Ha nincs következő játékos, a folyamat megáll.
3. Megnézi a két vizsgált felhasználó közül, melyikük áll régebb óta sorban: *maxWait*. Ezzel kiszámolja a maximum elo különbséget az adott esetre:

$$\text{FACTOR} = \frac{\text{MAX ELO RANGE} - \text{BASE ELO RANGE}}{\text{LONG WAIT TIME}}$$

$$\text{ELO RANGE} = \text{BASE ELO RANGE} + (\text{maxWait} \cdot \text{FACTOR})$$

4. Ha $\text{ELO DIFF} \leq \text{ELO RANGE}$ (ahol ELO DIFF = a két felhasználó közötti ELO különbség), akkor párosításra kerülnek. Egyéb esetben a ciklus átugorja a vizsgált játékost.

Összességében a rendszer működését a következőképpen lehet leírni: Sorba rendezve vizsgálja a queue-ban lévő játékosokat, és a queue-ban eltöltött idő mértékével nő a maximum elfogadható ELO különbség is, annak érdekében, hogy minél hamarabb meccset találjon a rendszer. A meccskeresési folyamat minden 2 másodpercben egyszer fut le.

Kódrészlet:

```
// === MATCHMAKING LOGIC CONSTANTS ===
const MAX_ELO_RANGE = 500;
const BASE_ELO_RANGE = 200;
const LONG_WAIT_TIME = 120;
const FACTOR = (MAX_ELO_RANGE - BASE_ELO_RANGE) / LONG_WAIT_TIME;
// =====
export function matchmake(queue) {
  queue = queue.sort((a, b) => a.elo - b.elo)
  const pairs = [];
  for (let i = 0; i < queue.length; i++) {
    const currPlayer = queue[i];
    const nextPlayer = queue[i + 1] ?? false;
    if (!nextPlayer) break;
    const maxWait = Math.max(currPlayer.waitTime, nextPlayer.waitTime)
    const eloRange = BASE_ELO_RANGE + (maxWait * FACTOR);
    const eloDiff = Math.abs(currPlayer.userElo - nextPlayer.userElo);
    if (eloDiff <= eloRange) {
      pairs.push([currPlayer, nextPlayer]);
      i++; // skip next player
    }
  }
  return pairs;
};
```

¹A Skill Based Matchmaking (képességalapú párosítás) olyan párosító rendszer, amit gyakran játékok használnak arra, hogy minden félnek élvezetes és kompetitív élményt nyújtsanak - egyik játékos se érezze, hogy hatalmas hátrányban van ellenfelével szemben.

Az algoritmus előnyei és hátrányai:
Előnyök:

- Gyors meccskeresés
- Alacsony erőforrás igény
- Garantálja, hogy a régóta várakozó játékosok előbb-utóbb találnak ellenfelet

Hátrányok:

- Mivel közvetlen szomszédokat vizsgál, előfordulhat, hogy jobb párosítás is létezik, mint amivel a rendszer végül előáll.
- Egy hosszú ideje várakozó játékos "lerontja" a meccskeresés eredményét, mivel nagyobb ELO különbséget enged meg.
- Csak ELO alapján dönt, nem figyel más paramétereket (winrate, pontosság)

8.4. Pontozás és rangsor

A pontozás az ELO rendszer segítségével történik. Ez a pontozási rendszer első sorban a sakkban terjedt el, de manapság már számos online játék használja. Az ELO rendszerben a pontok különbsége nagyságrendileg a következőket jelentik:

ELO különbség	Erősebb játékos nyeresi esélye
0	50%
100	64%
200	76%
400	90%
500	95%

A folyamat, lépésről lépésre:

1. Várt nyeresi esély kiszámítása: A rendszer alapja, hogy kiszámítsuk a várt nyeresi esélyét mindkét játékosnak. Ez a következőképpen kerül számításra:

$$ExpectedA = \frac{1}{1 + 10^{(RatingB - RatingA) / 400}} \quad (8.1)$$

2. ELO pontszám változás kiszámítása:

$$\Delta Rating = K \cdot (S_a - S_e) \quad (8.2)$$

Ahol K , úgynevezett K factor a rendszer érzékenységet jelenti. A SkillIssue rendszerében ez 30. Összehasonlításképpen: a FIDE² rendszerében a K értéke 10-40 között mozog, a játékos tapasztalatától függően.

3. Adatbázis módosítás: A kiszámított ELO (Rating) változás hozzáadódik a felhasználó jelenlegi pontszámához. Amennyiben az ELO változás értéke negatív, levonásra kerül.

Összegzés: Azért az ELO rendszer lett alkalmazva, mivel több iparágban is gyakran használt, jól kitapasztalt rendszer. Biztosítja, hogy amennyiben egy tapasztalatlanabb játékos játszik egy tapasztalt ellen, kevesebb lesz az ELO változása, ha veszít. Ugyanígy, ha egy gyenge játékos döntetlen mérkőzést játszik egy erős ellen, az eredmény ellenére növekedni fog az ELO-ja, az erősebb játékosnak akár ELO levonás is történhet ilyen esetben. Összességében, garantálja, hogy a rendszer fair, és minden játékost képességéhez mérten jutalmaz.

²Nemzetközi Sakkszövetség (Fédération Internationale des Échecs)

9. fejezet

Fejlesztési környezet és üzembe helyezés

9.1. Fejlesztői stack felállítása

A fejlesztői környezet felállításához szükséges a **Docker**, **Docker Compose**, illetve a **make** eszköz telepítése. Ez a szakasz a Linux-alapú rendszerekre történő telepítést mutatja be. Windows esetén a WSL használata javasolt.

1. Repository klónozása:

```
git clone https://github.com/Csaba44/SkillIssue.git
cd SkillIssue
```

2. Környezeti változók létrehozása:

Másoljuk le a `.env.example` fájlt `.env` névvel, majd töltsük ki a UID és GID értékeket, amelyek a host felhasználó azonosítói. Ezek szükségesek a fájljogosultságok szinkronizálásához a konténer és a host között.

```
cp .env.example .env
id -u && id -g
```

Majd hasonlóan, másoljuk le a `.env.dev.example` fájlt `.env.dev` névvel:

```
cp .env.dev.example .env.dev
```

3. Függőségek telepítése:

```
cd api && composer install
cd ../frontend && npm install
cd ../websocket && npm install
cd ..
```

4. Laravel alkalmazáskulcs generálása:

```
make dev-key
```

A parancs kiír egy `base64:...` előtaggal kezdődő kulcsot, amelyet a `.env.dev` fájlban az `APP_KEY` változóhoz kell rendelni.

5. Helyi host név hozzáadása:

A `/etc/hosts` fájlba fel kell venni a következő bejegyzést:

```
127.0.0.1    skillissue.local
```

WSL használata esetén a Windows host számítógépen a `C:/Windows/System32/drivers/etc/hosts` fájlba kell beleírni a fentebbi sort.

6. Fejlesztői stack elindítása:

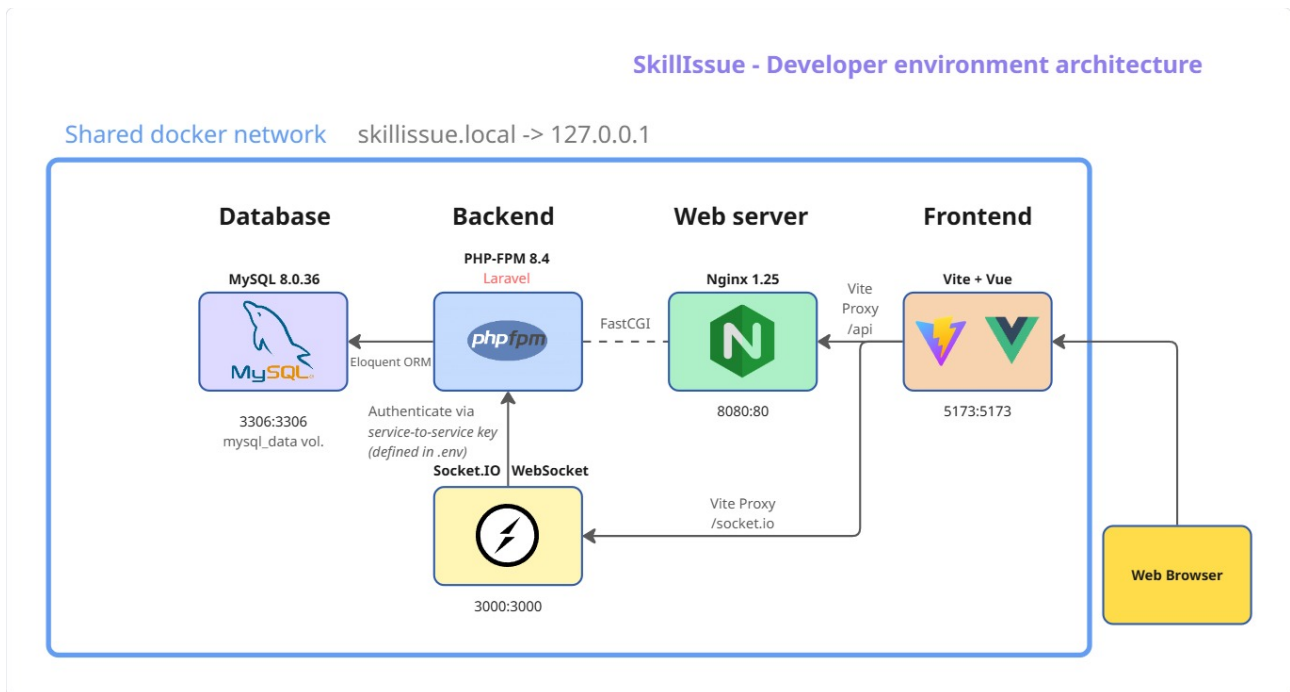
```
make dev-up
```

Az alkalmazás a `http://skillissue.local:5173` címen érhető el. A stack leállításához:

```
make dev-down
```

7. Migrációk futtatása (amennyiben nem futottak le automatikusan):

```
make migrate-seed
```



9.1. ábra. A SkillIssue fejlesztői környezetének architektúrája

Megjegyzés: Teszteléshez a SkillIssue weboldal is használható: <https://skillissue.hu>. Az alkalmazás jelszóval van védve:

Felhasználónév	skillissue
Jelszó	neumanneger2026

Az alkalmazáson belül előre létrehozott tesztfelhasználók is rendelkezésre állnak a különböző funkciók és jogosultságok kipróbálásához. Új fiók is regisztrálható, viszont az alábbi felhasználók is elérhetőek tesztelésre:

Szerepkör	E-mail cím	Jelszó
Adminisztrátor (Admin)	admin@gmail.com	password123
Általános felhasználó	gipsz.jakab@gmail.com	password123

9.2. Környezeti változók

A környezeti változók három `.env` fájlban vannak tárolva:

.env.dev Fejlesztői környezet környezeti változói

.env.prod Éles környezet környezeti változói

.env A Dockerhez használatos környezeti változók, a jogosultság szinkronizálására a Docker konténer és a host számítógép között.

Fontosabb Laravel környezeti változók:

Környezeti változó neve	Leírás
APP_ENV	Lehetséges értéke: local, production.
APP_KEY	Base64-el enkódolt kulcs, amit a Laravel alkalmazás titkosításra használ.
DB_*	A Laravel által létrehozott adatbázis-kapcsolat paraméterei.
SESSION_DRIVER	Pl.: cookie, database, file, stb. - Meghatározza, hol legyenek tárolva a session adatok.
SESSION_LIFETIME	Session élettartama, percben.
SESSION_ENCRYPT	Boolean, session adatok titkosított tárolása
SESSION_PATH	A session cookie milyen URL-en legyen elérhető (/ = egész domain).
SESSION_DOMAIN	Melyik domain-en legyen érvényes a session cookie.

Játékmenet és saját implementációkra vonatkozó környezeti változók:

Környezeti változó neve	Leírás
MAX_ROUNDS	Hány körös legyen a mérkőzés (Solo és Ranked)
SOLO_MAX_GUESS_TIME	Maximum válaszadási idő (másodperc, egyjátékos mód)
RANKED_MAX_GUESS_TIME	Maximum válaszadási idő (másodperc, Ranked 1 vs. 1 mód)
SERVICE_TOKEN	Service-to-service kommunikációhoz meghatározott közös token

A részletes dokumentáció megtalálható a A függelékben (Környezeti változók dokumentációja).

9.3. Fejlesztői vs. Éles konfiguráció

A konfigurációs különbségeket részletesen a 4.4. szakasz taglalja.

Összességében elmondható, hogy a **fejlesztői környezet** esetében az elsődleges szempont a könnyű elindítás, fejlesztés segítése, a hot-reload támogatása, parancssori eszközök (artisan) könnyű használatának biztosítása és a jogosultságok szinkronizálása. A fejlesztői környezetben az egységes munkavégzés érdekében **.prettierrc** Prettier konfigurációs fájl található, amely lehetővé teszi a kódbázis egységes formázását.

Az **éles környezetben** a fő szempont az egyszerű szállítás, deployment, illetve a biztonság.

9.4. Makefile parancsok

A hosszú parancsok kiküszöbölésének érdekében a Makefile segíti a fejlesztő munkáját. Hat részre van bontva: Dev, Migration, Build, Publish, Prod, Utils. A Makefile összes utasítása és részletes leírása: B függelék (Makefile parancsok referencia)

9.5. Tesztelési stratégia

A SkillIssue projekt tesztelési stratégiájának kialakításakor a szűkös időkeret és a kétfős csapatméret jelentette a legnagyobb kihívást. Bár az alkalmazás egy MVP (Minimum Viable Product), a komplex, valós idejű WebSocket események, az összetett statisztikai számítások és a kritikus állapotkezelés megkövetelte a tesztelési szintek kombinálását.

A tisztán manuális tesztelés helyett egy hibrid megközelítést alkalmaztunk. A backend üzleti logikáját (pl. statisztikák számítása, jogosultságkezelés) és a frontend legfontosabb komponenseit, illetve állapotkezelőit automatizált Unit és Feature tesztekkel védtük le. Ezzel párhuzamosan a valós idejű játékmenetet, a hálózati anomáliákat és a felhasználói élményt kiterjedt manuális teszteléssel validáltuk. Ez a stratégia biztosította, hogy az alapvető funkciók stabilak maradjanak a fejlesztés során, miközben az edge-case-ek is feltárásra kerültek.

9.6. Manuális tesztelés

A manuális tesztelési folyamat két jól elkülöníthető fázisban és módszertannal valósult meg a projekt során.

Fejlesztői tesztelés (WhiteBox megközelítés) Az első ágot a kódismereten alapuló tesztelés jelentette. Minden kódmódosítás, refaktorálás vagy új funkció implementálása után, a rendszerarchitektúra pontos ismeretének fényében vizsgáltuk az alkalmazást. A tesztelés során célzottan azokat a modulokat és API végpontokat terheltük, amelyekre az adott módosítások a legnagyobb eséllyel hatással lehettek. Ezzel a módszerrel a UI hibák és a hálózati deszinkronizációk nagy része már a fejlesztői környezetben kiküszöbölésre került.

Harmadik fél általi tesztelés (BlackBox megközelítés) A rendszer komplexitásából és a valós idejű játékmenetből adódóan a fejlesztői tesztelés önmagában nem volt elegendő. Bevontunk harmadik feleket (diáktársakat, barátokat) is a tesztelési folyamatba, akiknek az volt a feladatuk, hogy a kód bázis ismerete nélkül, átlagos felhasználóként navigáljanak az oldalon. Megpróbáltak lejátszani egyjátékos és rangsorolt meccseket, valamint váratlan interakciókkal "elrontani" a rendszert. Számos olyan hibára (például a böngészőlap meccs közbeni frissítése vagy a hálózati megszakadások kezelése) derült fény, amelyek javítása elengedhetetlen volt egy stabil alkalmazás kiadásához.

9.7. Automatizált tesztelés

Az automatizált tesztek feladata a regressziós hibák elkerülése, a kritikus backend üzleti logika hibátlan működésének garantálása, és a frontend állapotkezelésének validálása volt. A tesztek két fő részre bontottuk: a backend tesztek a PHPUnit keretrendszerrel, míg a frontend tesztek a Vitest és a Vue Test Utils segítségével készültek.

9.7.1. Backend tesztek (Laravel / PHPUnit)

A backend automatizált tesztjei elsősorban a jogosultságkezelésre, a mérkőzések érvényesítésére (JWT tokenek), valamint a felhasználói statisztikák (Win rate, Elo) pontos kiszámítására fókuszálnak.

9.7.2. Frontend tesztek (Vue 3 / Vitest)

A frontend tesztek célja a Pinia store-ok (állapotkezelés) és a Vue komponensek megfelelő renderelésének és interaktivitásának biztosítása.

Tesztfájl / Osztály	Típus	Leírás és vizsgált funkciók
<i>Autentikáció és Jogosultságok (Feature)</i>		
AuthTest	Feature	Regisztráció, bejelentkezés, helytelen jelszó kezelése, email validáció, kijelentkezés és a védett útvonalak (Auth middleware) tesztelése.
AdminAccessTest	Feature	Ellenőrzi, hogy csak az adminisztrátorok férhetnek hozzá a védett végpontokhoz (pl. tantárgyak létrehozása, felhasználók tiltása - Ban, meccsek törlése).
<i>Játékmenet és Működés (Feature)</i>		
VerifyAnswerTest	Feature	Komplex teszt a válaszok beküldésére. Validálja a többszörös JWT tokeneket (Question, Practice Session, Ranked, Service tokenek) és az adatbázis frissülését.
UserReportTest	Feature	Megbizonyosodik róla, hogy a felhasználók megfelelően tudnak mérkőzéseket és kérdéseket jelenteni érvényes azonosítókkal.
HealthTest	Feature	A rendszer alapvető elérhetőségét (Health check) vizsgálja.
<i>Üzleti Logika (Unit)</i>		
UserCalculationTest	Unit	A felhasználói statisztikák (Győzelmi arány, Win streak, Pontosság) és a rangok (Elo rendszer) számításának ellenőrzése. Vizsgálja a kitiltás (Ban) állapotának helyes lekérdezését is.

9.1. táblázat. Backend automatizált tesztek és azok funkciói

Tesztfájl / Osztály	Érintett Modul	Leírás és vizsgált funkciók
<i>Állapotkezelés és Útválasztás</i>		
UserStore.spec.js	Pinia Store	Munkamenet ellenőrzése (verifySession), sikeres bejelentkezés állapota. Kiemelten kezeli a 401-es hibát (automatikus kijelentkezés) és a WebSocket kapcsolat megfelelő felépítését, illetve bontását (logout esetén).
router.spec.js	Vue Router	Router Guard-ok tesztelése. Ellenőrzi, hogy az autentikálatlan felhasználók a login oldalra, a jogosulatlanok pedig az admin oldalról a főoldalra lesznek-e átirányítva.
<i>Komponensek és Interakciók</i>		
SoloWidget.spec.js	UI Komponens	Egyjátékos modul. Teszteli a kérdések és adatok helyes renderelését, a válaszadás gomb működését (emit), a hiányzó válasz esetén felugró hibaüzeneteket, és a jelentés megnyitását.
RankedWidget.spec.js	UI Komponens	Rangsorolt widget működése, a GameStore-ral való megfelelő integráció, valamint a válaszok helyes API beküldésének szimulációja.
QuestionReport.spec.js	UI Komponens	Kérdésjelentő űrlap tesztje: megjelenítés, üres mezők validációja (hibaüzenet-megjelenítés), valamint API hívások paramétereinek és a beküldés sikerességének (isSubmitted) ellenőrzése.

9.2. táblázat. Frontend automatizált tesztek és azok funkciói

10. fejezet

Összefoglalás

10.1. Megvalósított funkciók

Végeredményben a rendszer egy komplex kvízalkalmazást valósít meg, mely egyesíti a többjátékos kompetitív élményt oktatási funkciókkal. A rendszer központi elemei a valós idejű rangsorolt játékmenet, a fejlődés nyomonkövetését elősegítő rendszerek valamint az adminisztrációs rendszer.

A rendszer a következő funkciókat valósítja meg:

- Reszponzív viselkedésforma
- Regisztráció, Autentikáció
- Játékosprofilok
- ELO rangrendszer
- XP rendszer
- Daily streak (napi sorozat) számítás
- Winstreak (nyerési sorozat) számítás
- Egyjátékos gyakorló meccsek
- Többjátékos meccskeresési funkció (SBMM)
- 1 vs. 1 rangsorolt mérkőzések
- Játékos -és kérdés bejelentési funkciók
- Jogosultságrendszer
- Adminisztrációs rendszer

10.2. Nehézségek és tanulságok

A rendszer komplexitása miatt több helyen is szükség volt kreativitásra a problémák megoldása érdekében:

- **Egyszerű csalásra használható scriptek kiküszöbölése:** A lehető legminimálisabb információkat szabad közölni a klienssel, annak érdekében, hogy az automatizált csalásokat - amelyek a hálózati kéréseket figyelik - kiküszöbölje a rendszer.
- **Valós idejű játékmenet:** Edge case-ek kiküszöbölése: játékos nem fogadja el a meccset, kilép, visszatapcsol, több lapon használja az alkalmazást.
- **Meccskeresés:** Fair, jól balance-olt rendszer kialakítása, minimális erőforrásigénnyel.
- **Tesztelés:** A sok funkció manuális tesztelése nehézkes volt, ami rávilágított az automatizált tesztek fontosságára.

10.3. Továbbfejlesztési lehetőségek

A rendszer jelenlegi állapota egy MVP-t (Minimum viable product) valósít meg. A rendszer számos továbbfejlesztési lehetőséggel rendelkezik, amelynél a fő szempontok a következők: csalás kiküszöbölése, SBMM továbbfejlesztése, játékmódok bővítése.

Néhány példa továbbfejlesztési lehetőségekre:

- **Csalás kiküszöbölése:** Rangsorolt meccs folyamán több adat rögzítése, annak érdekében, hogy könnyebben eldönthető lehessen, hogy csalt-e a játékos. Például: Focus loss detection (játékos elhagyja-e az oldalt meccs közben - "tab-out"), pontosabb guess time számítás.
- **Új játékmódok:** Olyan játékmódok bevezetése, amelyek elősegítik a tanulást: CO-OP játékmód, ahol több játékos együttes munkával megismer egy adott érettségi témát; csoportos kompetitív játékmódok.
- **Kérdések bővítése:** Olyan közösségi felület, ahol a játékosok kérdéseket javasolhatnak.
- **Mobilapplikáció:** Android, iOS alkalmazás fejlesztése a jobb felhasználói élmény érdekében, például Flutter segítségével.
- **Badge és Achievement rendszer:** Bizonyos mérföldkövek után a játékos Badge-eket kap, amelyek megjelennek a profiljukon, ezzel növelve a motivációt. (A Badge rendszer adatbázis struktúrája megtalálható az MVP-ben, de a implementálásra nem kerül)
- **Távoli jövő - teljes érettségi felkészítő platform:** Monetizált érettségi felkészítő platform, ahol a jelenleg ismertetett funkciókon kívül megtalálható hivatalos tanári segítség és több erőforrás a tanuló felkészülésének elősegítése érdekében.

A. függelék

Környezeti változók dokumentációja

A rendszer konfigurációja környezeti változókon keresztül történik, amelyeket a `.env` fájlok tartalmaznak. Külön fájl szolgál a Docker környezethez (`.env`), a fejlesztéshez (`.env.dev`) és az éles üzemhez (`.env.prod`).

Változó	Alapértelmezett / Példa	Leírás
<i>Docker (Alapkonfiguráció)</i>		
UID / GID	1000	A gazdagép felhasználói és csoport azonosítója a fájljogosultságokhoz.
<i>Alkalmazás (Laravel)</i>		
APP_NAME	SkillIssue	Az alkalmazás megnevezése.
APP_ENV	local / production	A futtatási környezet típusa.
APP_KEY	base64:...	Az alkalmazás titkosító kulcsa (<code>make dev-key</code>).
APP_URL	https://skillissue.hu	Az alkalmazás elérési útvonala.
APP_DEBUG	true / false	Hibakeresési mód ki- és bekapcsolása.
<i>Adatbázis (MySQL)</i>		
DB_CONNECTION	mysql	Az adatbázis-kezelő típusa.
DB_HOST	mysql	Az adatbázis-szolgáltatás gépneve (Docker-en belül).
DB_DATABASE	skillissue	A használt adatbázis neve.
DB_USERNAME	root / appuser	Az adatbázis eléréséhez használt felhasználónév.
DB_PASSWORD	*****	Az adatbázis jelszava.
<i>Játékmechanika</i>		
MAX_ROUNDS	5	Egy mérkőzés maximális köreinek száma.
SOLO_MAX_GUESS_TIME	30	Válasadási idő másodpercben (gyakorló mód).
RANKED_MAX_GUESS_TIME	30	Válasadási idő másodpercben (rangsorolt mód).
VITE_*	...	A frontend számára is elérhetővé tett változók (prefixelt).
<i>Biztonság és Kommunikáció</i>		
SERVICE_TOKEN	abcdefg123456	A belső szolgáltatások (API és WebSocket) közötti hitelesítéshez.
SANCTUM_STATEFUL_DOMAINS	skillissue.hu, ...	Engedélyezett domainek a sütialapú hitelesítéshez.

A.1. táblázat. A projektben használt legfontosabb környezeti változók

Fontos: Éles környezetben (`.env.prod`) a `SESSION_ENCRYPT` értéke kötelezően `true`, a `LOG_LEVEL` pedig `warning` szintre van állítva a biztonság és a teljesítmény érdekében.

B. függelék

Makefile parancsok referencia

A projekt gyökérkönyvtárában található `Makefile` segítségével egyszerűsíthetők a gyakran ismételt Docker, Laravel és karbantartási feladatok. Az alábbi táblázat összefoglalja a rendelkezésre álló parancsokat és azok funkcióit.

Parancs	Futtatott művelet	Leírás
<i>Fejlesztői környezet (DEV)</i>		
<code>make dev-key</code>	<code>docker compose run artisan key:generate</code>	Új Laravel <code>APP_KEY</code> generálása a fejlesztői környezethez.
<code>make dev-up</code>	<code>docker compose up --build</code>	Konténerek indítása az előtérben, kényszerített újraépítéssel.
<code>make dev-up-d</code>	<code>docker compose up -d</code>	Konténerek indítása a háttérben (detached mód).
<code>make dev-down</code>	<code>docker compose down</code>	Fejlesztői konténerek leállítása.
<code>make dev-down-v</code>	<code>docker compose down -v</code>	Leállítás és az adatbázis volume-ok teljes törlése.
<i>Adatbázis műveletek (MIGRATE)</i>		
<code>make migrate</code>	<code>./artisan migrate</code>	Új adatbázis migrációk futtatása.
<code>make migrate-seed</code>	<code>./artisan migrate --seed</code>	Migrálás és alapértelmezett adatok feltöltése.
<code>make migrate-fresh</code>	<code>./artisan migrate:fresh</code>	Összes tábla törlése és migrációk újrafuttatása.
<code>make migrate-fresh-seed</code>	<code>./artisan migrate:fresh --seed</code>	Teljes alaphelyzetbe állítás tesztadatok betöltésével.
<i>Build & Publish</i>		
<code>make build</code>	<code>docker build -t ...</code>	Éles Docker image-ek összeállítása (opcionális verziózással).
<code>make push</code>	<code>docker push ...</code>	Elkészült image-ek feltöltése a Docker Hub tárhelyre.
<code>make release</code>	<code>make build push</code>	Verziózott kiadás gyártása és publikálása egy lépésben.
<i>Éles környezet (PROD)</i>		
<code>make prod-up</code>	<code>docker compose up</code>	Éles környezet indítása az előtérben.
<code>make prod-up-d</code>	<code>docker compose up -d</code>	Éles környezet indítása a háttérben.
<code>make prod-pull</code>	<code>docker compose pull</code>	Legfrissebb image-ek letöltése a Docker Hub-ról.
<code>make prod-db-seed</code>	<code>docker compose run artisan db:seed</code>	Éles adatbázis inicializálása (csak egyszeri futtatásra).
<i>Segédprogramok (UTILS)</i>		
<code>make logs</code>	<code>docker compose logs -f</code>	Éles naplófájlok (<i>logs</i>) folyamatos követése.
<code>make ps</code>	<code>docker compose ps</code>	Futó konténerek és állapotuk listázása.
<code>make prettier</code>	<code>npx prettier --write ...</code>	Forráskód automatikus formázása (Front-end és Websocket).

B.1. táblázat. A projektben használt automatizációs parancsok listája

Megjegyzés: A táblázatban szereplő `migrate` parancsok közvetlenül a gazdagépen futtatott `./artisan` scriptet használják, amely a megfelelő Docker konténerben hajtja végre a műveletet.